LLM & Agentic & Thinking

Intro

- Until now we have seen how to improve models by training
  - with bigger model size: add depth and width to transformer/SSM/…
  - on more data
- One more axis of improvement is adding more test-time compute
- Related to human thinking: (Kahneman 2013, „Think, Fast and Slow”)
  - Fast thinking (S1): operate quickly and automatically
  - Slow thinking (S2): longer thought process, deliberation, reflection, mental energy
- How can we emulate this dimension of thinking in neural networks?
- Other tricks to make LLMs reason better & produce better answers?

Test-Time Compute: Few-Shot

- Example: Given as input to LLM
  Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?
  A: The answer is 11.
  Q: I have 3 apples, my dad has 2 more apples than me, how many apples do we have in total?
  A:

- LLM answer: I have 3 apples, my dad has 2 more apples than me, so he has 5 apples. 3+5=8, We have 8 apples in total

- Increase test-time compute by letting model consume more input tokens

- Related questions/answers allow model to draw analogous conclusions

- But beware of:

  - Recency bias: Repeat last answer to current question

  - Majority label bias: If most examples show one class of output, then model may just take that one

  - Order sensitivity: Output can vary with order of few-shot examples

  - Prompt length limitation: If too many examples, not all will fit in prompt

  - Position bias: Positional encoding might make earlier examples less relevant to output

Test-Time Compute: CoT

- Example: Q: I have 3 apples, my dad has 2 more apples than me, how many apples do we have in total? A:
  - S1 answer: 8
  - S2 answer: I have 3 apples, my dad has 2 more apples than me, so he has 5 apples. 3+5=8, We have 8 apples in total
- S2 takes more test-time compute than S1
  - Increase compute by generating more output-tokens
- S2 gives a Chain-of-Though trace
- How to elicit CoT:
  - Zero-shot: Add „Think step by step” or similar to answer
  - Few-shot: Add related questions with CoT trace to prompt
  - Fine-tuning on dialogues with CoT traces
  - RL-learning by allowing it to add more reasoning traces and supervise only on final correct answer

Test-Time Compute: Tree of Thought

- „Tree of Thoughts: Deliberate Problem Solving with Large Language Models”, Yao et al 2023
  - Explore multiple possibilities at each step
  - Search process can be breadth first or depth first search

Algorithm 1 ToT-BFS($x, p_\theta, G, k, V, T, b$)

Require: Input x, LM p_θ, thought generator $G()$ & size limit k, states evaluator $V()$, step limit T, breadth limit b.

$S_0 \leftarrow \{x\}$
for $t = 1, \cdots, T$ **do**
  $S'_t \leftarrow \{[s, z] \mid s \in S_{t-1}, z_t \in \mathrm{G}(p_\theta, s, k)\}$
  $V_t \leftarrow V(p_\theta, S'_t)$
  $S_t \leftarrow \arg\max_{S \subset S'_t, |S|=b} \sum_{s \in S} V_t(s)$
end for
return $G(p_\theta, \arg\max_{s \in S_T} V_T(s), 1)$

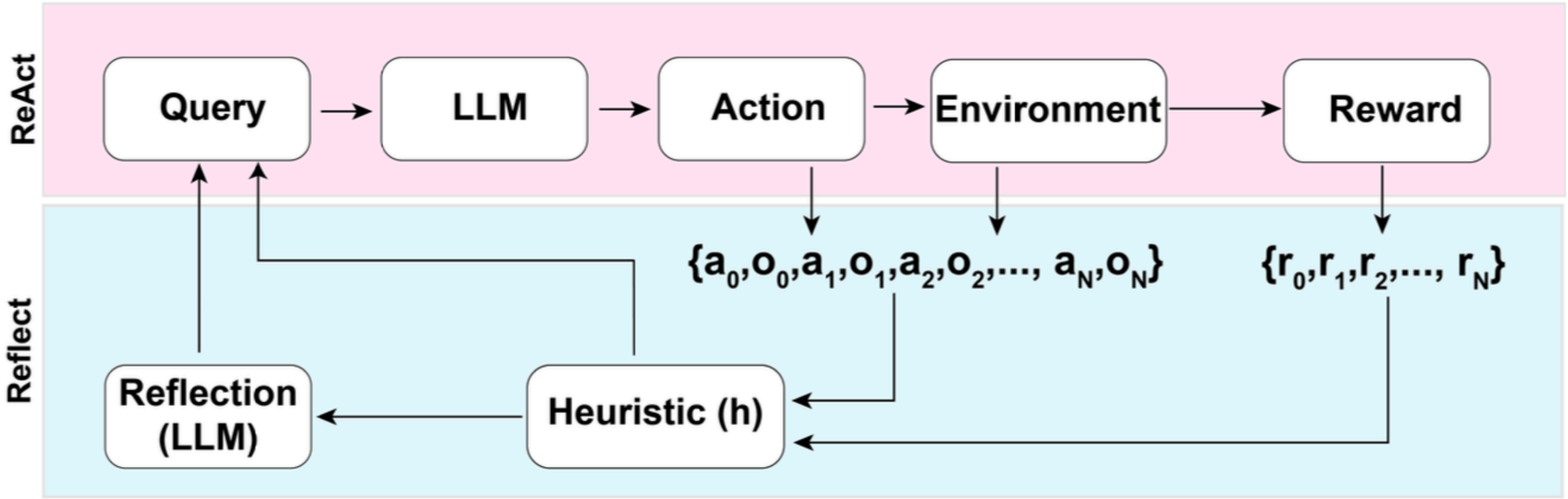
Algorithm 2 ToT-DFS($s, t, p_\theta, G, k, V, T, v_{th}$)

Require: Current state s, step t, LM p_θ, thought generator $G()$ and size limit k, states evaluator $V()$, step limit T, threshold v_{th}

if $t > T$ **then** record output $G(p_\theta, s, 1)$
end if
for $s' \in G(p_\theta, s, k)$ **do** ▷ sorted candidates
  if $V(p_\theta, \{s'\})(s) > v_{thres}$ **then** ▷ pruning
    DFS($s', t+1$)
  end if
end for

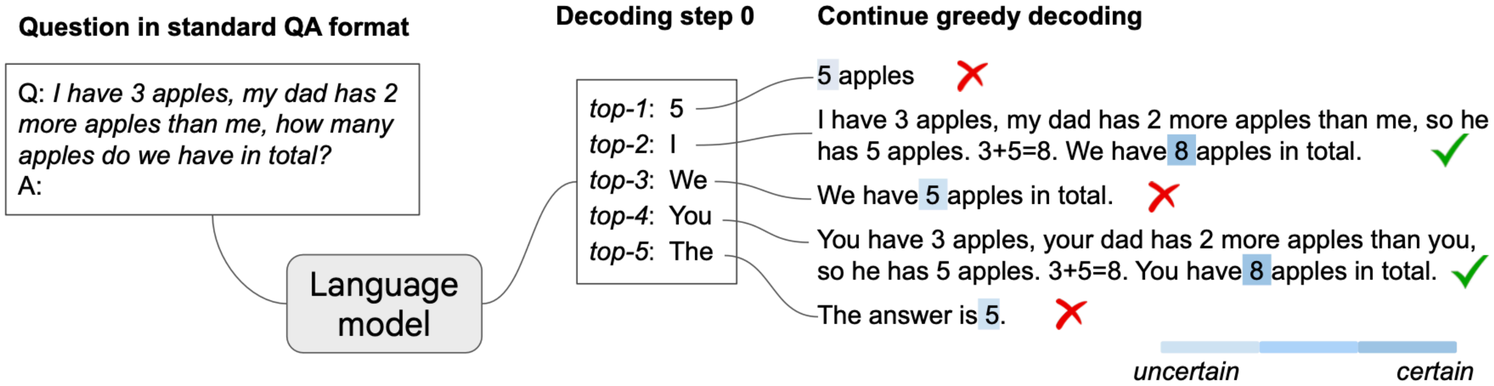
Test-Time Compute: Reflection

- „Reflexion: Language Agents with Verbal Reinforcement Learning", Shinn et al 2023
  - Add dynamic memory and self-reflection capabilities
  - Used in LLMs in some environment (text game, coding, …)
  - Add heuristic that may optionally decide to reset the generation based on reflecting current state



Test-Time Compute: Parallel Sampling

- Example: Q: I have 3 apples, my dad has 2 more apples than me, how many apples do we have in total? A:
  - LLM answer 1: 8
  - LLM answer 2: 7
  - LLM answer 3: 8
  - Majority vote: 8



- How can we sample effectively in parallel?
  - Sample with temperature parameter $T > 0$
  - Branch at first token: „Chain-of-Thought Reasoning Without Prompting" Wang & Zhou, 2024:
    - Sample different first tokens, then decode with temperate $T = 0$ greedily -> gives diverse answers

Test-Time Compute: Sequential Revision

- Q: I have 3 apples, my dad has 2 more apples than me, how many apples do we have in total?
  A: The answer is 5
  Incorrect, review your previous answer and find problems with your answer.
  A: Upon reviewing my answer, I recognize a mistake. I incorrectly concluded that we have 5 apples in total, even though only dad has 5.
  Based on the problems you found, improve your answer.
  A: Therefore, the answer will be 8.

- Does not seem to work yet: „Large Language Models Cannot Self-Correct Reasoning Yet", Huang et al, 2024

  - Possibly need to be trained to revise?

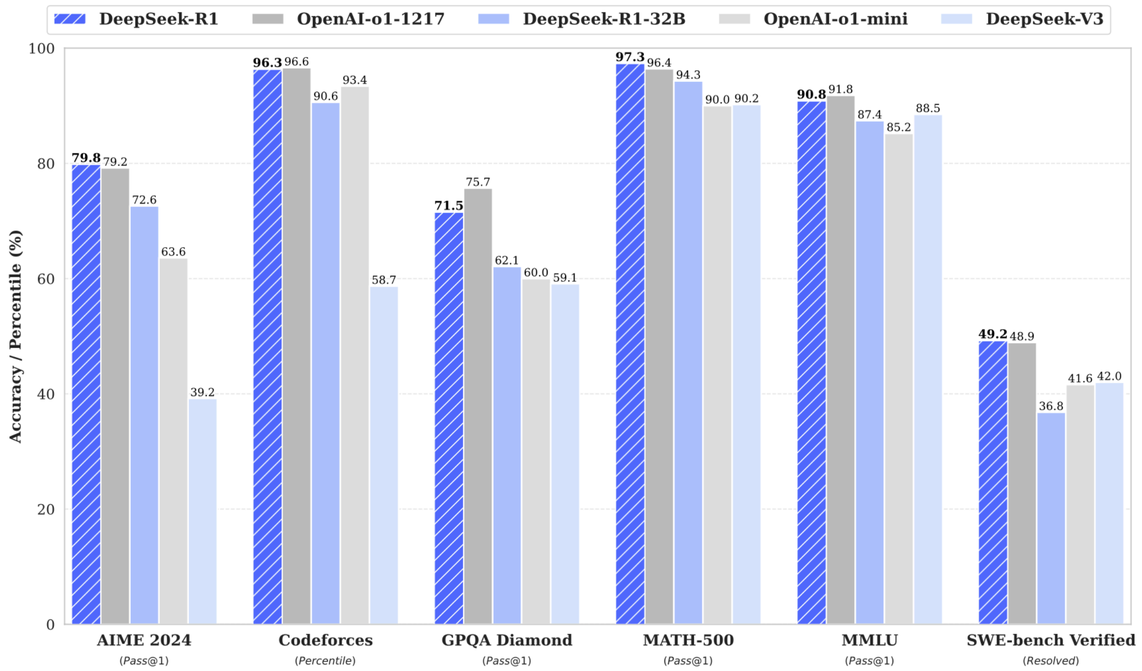
		# calls	GSM8K	CommonSenseQA	HotpotQA
GPT-3.5	Standard Prompting	1	**75.9**	**75.8**	**26.0**
	Self-Correct (round 1)	3	75.1	38.1	25.0
	Self-Correct (round 2)	5	74.7	41.8	25.0
GPT-4	Standard Prompting	1	**95.5**	**82.0**	**49.0**
	Self-Correct (round 1)	3	91.5	79.5	**49.0**
	Self-Correct (round 2)	5	89.0	80.0	43.0

Test-Time Compute: RL For Reasoning

- Use a collection of questions with ground truth answers
  - STEM problems and puzzles with verifiers
  - Coding problems with unit tests
- Format for reasoning:
  - Enclose reasoning in <think> … </think> segments
  - Final answer is given in <answer> … </answer> segment
- RL reward on correct format and correct answer
- OpenAI o-series models pioneered reasoning
- DeepSeek-R1 followed up with a similar model

Test-Time Compute: DeepSeek-R1

- Cold-Start SFT: Fine tune DeepSeek-V3-base (a non-reasoning LLM) on thousands of CoT traces
    - Without it, RL reasoning still works but gives garbled traces (mixed languages and readability)
- RL training phase with formatting and accuracy reward trained with GRPO
- Rejection sampling + non-reasoning SFT:
    - Sample reasoning answers after first RL stage
    - Filter out CoTs with mixed languages, long paragraphs, and code blocks
    - Add non-reasoning answers by sampling from DeepSeek-V3-base
    - Fine-tune the DeepSeek-V3-Base on the total 800k samples for 2 epochs.
- 2nd RL training phase on both reasoning and non-reasoning with additional helpfulness, harmlessness and reasoning rewards



Test-Time Compute: DeepSeek-R1

- Takeaways:
  - Pure RL model still works, has backtracking and reflection capabilities
  - Has been replicated (Open-R1, SimleRL-reason, TinyZero etc.)
  - Reasoning traces are not always helpful and correct
    - Is some LLM reasoning hidden and not directly corresponding to reasoning trace?
- Things that did not work:
  - Process reward models (PRM): Supervise reasoning step-by-step
  - Monte-Carlo-Tree-Search (MCTS): Too large search space, never got good signal

Test-Time Compute: Tool-Use

- Offload some tasks to specialized programs:
  - Web search
  - Python interpreter
  - …
- Done in o3 & o4-mini from OpenAI
- „PAL: Program-aided Language Models", Gao et al 2023
  - LLM is prompted to generate program
  - All other text is python comment
  - After LLM generated program, it is executed
  - Prompts are hand-generated

CoT

Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 tennis balls. 2 cans of 3 tennis balls each is 6 tennis balls. 5 + 6 = 11. The answer is 11.

Q: The bakers at the Beverly Hills Bakery baked 200 loaves of bread on Monday morning. They sold 93 loaves in the morning and 39 loaves in the afternoon. A grocery store returned 6 unsold loaves. How many loaves of bread did they have left?

Model Output

A: The bakers started with 200 loaves. They sold 93 in the morning and 39 in the afternoon. So they sold 93 + 39 = 132 loaves. The grocery store returned 6 loaves. So they had 200 - 132 - 6 = 62 loaves left.
The answer is 62.

PAL

Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 tennis balls.
`tennis_balls = 5`
2 cans of 3 tennis balls each is
`bought_balls = 2 * 3`
tennis balls. The answer is
`answer = tennis_balls + bought_balls`

Q: The bakers at the Beverly Hills Bakery baked 200 loaves of bread on Monday morning. They sold 93 loaves in the morning and 39 loaves in the afternoon. A grocery store returned 6 unsold loaves. How many loaves of bread did they have left?

Model Output

A: The bakers started with 200 loaves
`loaves_baked = 200`
They sold 93 in the morning and 39 in the afternoon
`loaves_sold_morning = 93`
`loaves_sold_afternoon = 39`
The grocery store returned 6 loaves.
`loaves_returned = 6`
The answer is
`answer = loaves_baked - loaves_sold_morning - loaves_sold_afternoon + loaves_returned`

```
>>> print(answer)
74
```

Test-Time Compute: Tool-Use

- Offload some tasks to specialized programs:
  - Web search
  - Python interpreter
  - …
- Done in o3 & o4-mini from OpenAI
- „PAL: Program-aided Language Models", Gao et al 2023
  - LLM is prompted to generate program
  - All other text is python comment
  - After LLM generated program, it is executed
  - Prompts are hand-generated

CoT

Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 tennis balls. 2 cans of 3 tennis balls each is 6 tennis balls. 5 + 6 = 11. The answer is 11.

Q: The bakers at the Beverly Hills Bakery baked 200 loaves of bread on Monday morning. They sold 93 loaves in the morning and 39 loaves in the afternoon. A grocery store returned 6 unsold loaves. How many loaves of bread did they have left?

Model Output

A: The bakers started with 200 loaves. They sold 93 in the morning and 39 in the afternoon. So they sold 93 + 39 = 132 loaves. The grocery store returned 6 loaves. So they had 200 - 132 - 6 = 62 loaves left.
The answer is 62.

PAL

Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 tennis balls.
`tennis_balls = 5`
2 cans of 3 tennis balls each is
`bought_balls = 2 * 3`
tennis balls. The answer is
`answer = tennis_balls + bought_balls`

Q: The bakers at the Beverly Hills Bakery baked 200 loaves of bread on Monday morning. They sold 93 loaves in the morning and 39 loaves in the afternoon. A grocery store returned 6 unsold loaves. How many loaves of bread did they have left?

Model Output

A: The bakers started with 200 loaves
`loaves_baked = 200`
They sold 93 in the morning and 39 in the afternoon
`loaves_sold_morning = 93`
`loaves_sold_afternoon = 39`
The grocery store returned 6 loaves.
`loaves_returned = 6`
The answer is
`answer = loaves_baked - loaves_sold_morning - loaves_sold_afternoon + loaves_returned`

```
>>> print(answer)
74
```

Test-Time Compute: Tool Use

- „Chain of Code: Reasoning with a Language Model-Augmented Code Emulator", Li et al 2024
- Output is pure python program
- If line of output cannot be executed, let LLM try to update program state

Chain of Code (Ours)

```
Q: How many countries have I been to?  I've been to Mum-
bai, London, Washington, Grand Canyon, Baltimore, ...
1    places, countries = ["Mumbai", ...], set()
delta state:  {places = ['Mumbai', ...], countries =
set()}
2    for place in places:
delta state:  {place = 'Mumbai'}
3      country = get_country(place)
delta state:  {country = 'India')}
4      countries.add(country)
delta state:  {countries = {'India'}}
5    answer = len(countries)    delta state:  {answer =
52}
A: 52 (100%, ✓)
```

Test-Time Compute: Tool Use

- „ReAct: Synergizing Reasoning and Acting in Language Models”, Yao et al 2022
- Use search engine to inject facts into output
- Use reasoning traces to help process facts and propose new search queries
- In-context samples for learning ReACT

(1) Hotspot QA

Question: Aside from the Apple Remote, what other device can control the program Apple Remote was originally designed to interact with?

(1d) ReAct (Reason + Act)

Thought 1: I need to search Apple Remote and find the program it was originally designed to interact with.
Act 1: Search[Apple Remote]
Obs 1: The Apple Remote is a remote control introduced in October 2005 by Apple … originally designed to control the Front Row media center program …

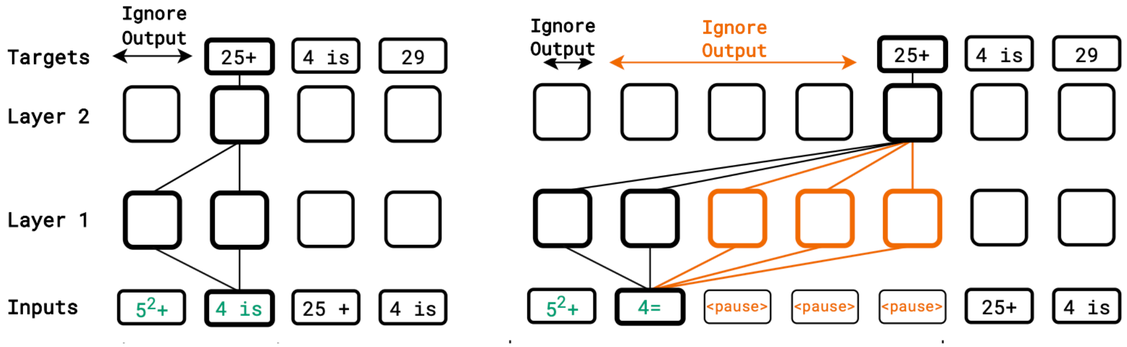
Thought 2: Apple Remote was originally designed to control the Front Row media center program. I need to search Front Row next and find what other device can control it.
Act 2: Search[Front Row]
Obs 2: Could not find [Front Row]. Similar: ['Front Row Seat to Earth', 'Front Row Motorsports',' Front Row (software)', …]

Thought 3: Front Row is not found. I need to search Front Row (software) .
Act 3: Search[Front Row (software)]
Obs 3: Front Row is a discontinued media center software …

Thought 4: Front Row (software) is controlled by an Apple Remote or the keyboard function keys. So the answer is keyboard function keys.
Act 4: Finish[keyboard function keys]

Test-Time Compute: Thinking Tokens

- „Thinking Tokens for Language Modeling", Herel & Mikolov 2024: Give more output tokens to increase computation
  - Add special token <T>: Does not have meaning
  - They do not carry any meaning
  - Add token to output and fine-tune
- „Think Before You Speak: Training Language Models With Pause Tokens", Goyal et al 2024: Give more input tokens to increase computation
  - Append dummy tokens (. or #)
  - Finetune on dataset.



(a) Standard inference and finetuning (b) Pause-inference and finetuning

Test-Time Compute: Train CoT

- „STaR: Self-Taught Reasoner Bootstrapping Reasoning With Reasoning", Zelikman et al, 2022
  - How to get good CoTs for training LLMs?
  - Generating CoTs can result in bad CoTs -> We condition them only on input prompt
  - Instead, start CoTs backwards by conditioning them on input prompt and ground truth answer
  - We can get CoTs for hard examples

Algorithm 1 STaR

Input M: a pretrained LLM; dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{D}$ (w/ few-shot prompts)

$M_0 \leftarrow M$ # Copy the original model
for n **in** $1 \ldots N$ **do** # Outer loop
  $(\hat{r}_i, \hat{y}_i) \leftarrow M_{n-1}(x_i) \quad \forall i \in [1, D]$ # Perform rationale generation
  $(\hat{r}_i^{\text{rat}}, \hat{y}_i^{\text{rat}}) \leftarrow M_{n-1}(\text{add_hint}(x_i, y_i)) \quad \forall i \in [1, D]$ # Perform rationalization
  $\mathcal{D}_n \leftarrow \{(x_i, \hat{r}_i, y_i) \mid i \in [1, D] \wedge \hat{y}_i = y_i\}$ # Filter rationales using ground truth answers
  $\mathcal{D}_n^{\text{rat}} \leftarrow \{(x_i, \hat{r}_i^{\text{rat}}, y_i) \mid i \in [1, D] \wedge \hat{y}_i \neq y_i \wedge \hat{y}_i^{\text{rat}} = y_i\}$ # Filter rationalized rationales
  $M_n \leftarrow \text{train}(M, \mathcal{D}_n \cup \mathcal{D}_n^{\text{rat}})$ # Finetune the original model on correct solutions - inner loop
end for

Test-Time Compute: Memory

- Sensory Memory:
  - Earliest stage
  - Lasts for a few seconds, retain visual auditory information
- Short-Term Memory or Working Memory:
  - Store information that we are currently aware of
  - Needed to carry out complex cognitive tasks
  - Lasts for 20-30 seconds
- Long-Term Memory:
  - Store information for decades
  - Explicit/declarative memory: Facts and events, can be consciously recalled
  - Implicit/procedural memory: Unconscious, involves skills (riding a bike, swimming, …)

- Sensory Memory:
  - Corresponds to learning embeddings of ran inputs
- Short-Term Memory or Working Memory:
  - In-context memory
  - Restricted by finite context window of LLMs
- Long-Term Memory:
  - External vector store
  - Can be accessed via retrieval

Test-Time Compute: Memory

- Maximum Inner Product Search (MIPS):
  - Save embeddings in vector store database
  - Retrieve embeddings via MIPS
  - For fast retrieval use Approximate nearest neighbors (ANN) search to retrieve top-k neighbors
    - Lose little accuracy in practice / get high speedup
- Also known as RAG: Retrieval Augmented Generation